

I2C-Based Game Controller Interface in System-on-Chip Architecture

Yashaswini S

Department of Electronics and Communication Engineering
Cambridge Institute of Technology
Bengaluru, Karnataka, India
yashaswini.23ece@cambridge.edu.in

Sanjeitha K S

Department of Electronics and Communication Engineering
Cambridge Institute of Technology
Bengaluru, Karnataka, India
sanjeitha.23ece@cambridge.edu.in

Vindhya D

Department of Electronics and Communication Engineering
Cambridge Institute of Technology
Bengaluru, Karnataka, India
vindhya.23ece@cambridge.edu.in

V M Swathika

Department of Electronics and Communication Engineering
Cambridge Institute of Technology
Bengaluru, Karnataka, India
swathika.23ece@cambridge.edu.in

Dr. Priya Singh

Department of Electronics and Communication Engineering
Cambridge Institute of Technology
Bengaluru, Karnataka, India
priya.ece@cambridge.edu.in

Mr. Tharun Reddy

Design and Verification Engineer
Elevium – by Nanochip
External Guide

Abstract—In this work, a game controller interface based on the I2C protocol is designed using a System-on-Chip architecture. The main idea is to read button inputs from an external controller and store them inside the system. A finite state machine is used to control the communication between the master and the controller.

The design is implemented using Verilog HDL and tested using simulation. From the results, it is observed that the data transfer is happening correctly and the communication follows the expected I2C sequence.

This design provides a simple way to connect controller inputs in embedded systems and can be extended further for other applications.

Index Terms—I2C protocol, System-on-Chip, game controller interface, Verilog HDL, finite state machine, embedded systems, hardware verification.

I. INTRODUCTION

Embedded systems are used in many applications such as gaming devices, control systems, and portable electronics. In these systems, it is important to connect external input devices like controllers in a simple and reliable way. For this purpose, communication protocols are used to transfer data between devices.

Among different protocols, the Inter-Integrated Circuit (I2C) protocol is commonly used because it requires only two lines for communication and supports multiple devices on the same bus. It is simple to implement and is widely used in embedded and System-on-Chip applications.

In this work, an I2C based game controller interface is designed as part of a SoC architecture. The main aim is to read button input data from a controller and store it

inside the system for further processing. The communication is controlled using a finite state machine, which manages different stages such as start condition, address transmission, acknowledgement checking, and data reception.

The design is implemented using Verilog HDL and verified through simulation. The waveform results are analyzed to check whether the communication is happening correctly. The proposed system provides a simple way to interface controller inputs with SoC based systems.

This work can be extended in future to support more devices or to integrate with other communication protocols and hardware platforms.

A. Contributions of the Proposed Work

The main contributions of this work are summarized as follows:

- Design and implementation of an I2C based game controller interface integrated within a System-on-Chip architecture.
- Development of a modular Verilog HDL-based hardware framework for controller communication and data handling.
- Integration of an I2C master block with internal status registers for capturing and storing controller button-state data
- Simulation-based functional verification of the proposed design to confirm correct protocol operation and system reliability.

- Evaluation of the architecture in terms of communication latency and overall interface efficiency.

II. LITERATURE SURVEY

I2C communication has been widely used in embedded systems for connecting different devices using only two lines. Many researchers have implemented I2C controllers and studied their behavior in different environments.

In [1], a verification method for an I2C master controller is discussed. The main focus of this work is on testing and checking whether the communication is happening correctly.

An implementation of an I2C master on FPGA using Verilog HDL is presented in [2]. The design works well for communication, but it is mainly limited to a single module and does not consider full system integration.

The work in [3] explains how I2C communication can be modeled between different parts of a system. Although this helps in understanding system-level communication, it makes the design slightly complex.

In [4], an I2C slave interface is designed using Verilog. The design ensures proper data transfer and synchronization, but it focuses only on the slave side.

Another work in [5] uses I2C for EEPROM interfacing on FPGA. This shows that I2C can be used for reliable data transfer, but the application is limited to memory devices.

A comparison between I2C and SPI protocols is given in [6], where it is shown that I2C is suitable for simple and low-speed communication.

In [7], an interface between I2C and APB is developed for SoC systems. This helps in connecting different modules, but it increases the design complexity.

From all these works, it can be understood that most of the designs focus only on individual parts like master, slave, or protocol verification. Very few works combine everything into a complete system. So, in this project, an attempt is made to design an I2C-based game controller interface as part of a System-on-Chip using Verilog HDL.

III. SYSTEM ARCHITECTURE

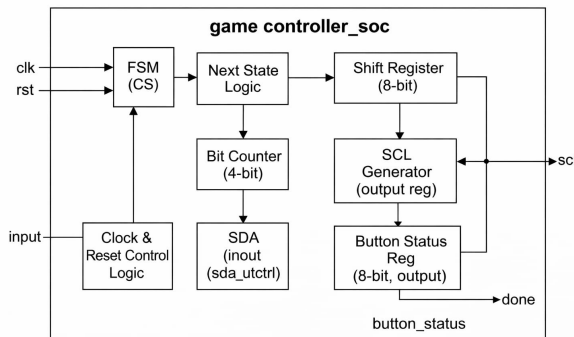


Fig. 1. Block diagram of the I2C-based game controller interface

The proposed architecture comprises several interconnected functional blocks that collectively implement I2C-based communication between the game controller and the SoC. Each block is assigned a dedicated function to support synchronized data transfer, control sequencing, and storage of the received button-state information.

1. Clock and Reset Control Logic

This block provides the timing (*clk*) and reset (*rst*) signals. The clock ensures synchronous operation of all sequential elements, while the reset initializes the system to a defined state during startup or reconfiguration.

2. Finite State Machine (FSM)

The finite state machine governs the transaction flow of the I2C interface. It manages the progression through idle, address transmission, data reception, acknowledgment handling, and transaction completion states.

3. Next State Logic

This logic determines the transition from one FSM state to another based on the current state and input conditions. It ensures that communication proceeds in a valid sequence according to the I2C protocol.

4. Bit Counter (4-bit) The bit counter tracks the number of bits transmitted or received during each communication phase. It is used to maintain correct timing and to ensure that full-byte transfers are completed accurately.

5. SDA Line

The serial data line serves as the bidirectional data path in the I2C bus. It enables the master and slave to exchange address, control, and button state information using serial communication.

6. Shift Register (8-bit)

The 8-bit shift register supports serial-to-parallel and parallel to serial conversion. Incoming bits from the controller are accumulated in this register, and outgoing data is serialized when required.

7. SCL Generator

The serial clock line generator produces the clock pulses required for I2C synchronization. It coordinates the timing of data sampling and shifting between the master and slave devices.

8. Button Status Register The button status register stores the received 8 bit controller input data. This register provides internal access to the captured button states and may also be mapped to GPIO outputs for observation or further processing.

IV. METHODOLOGY

The proposed SoC interface is developed using a modular design approach in Verilog HDL. The methodology is organized into requirement definition, module implementation, system integration, simulation, and verification.

A. Requirement Specification

The system is required to initiate I2C read transactions from an external controller modeled as a slave device. The master must be configurable through an APB interface, and

the received button state data must be stored in internal registers or made available through GPIO outputs. The controller emulator must respond with predefined button values during read operations.

- The I2C Master must initiate read operations from an external I2C slave device representing the controller.
- The I2C Master should be programmable through an APB (Advanced Peripheral Bus) interface.
- Button states received from the controller must be stored in internal status registers or mapped to GPIO outputs.
- The controller emulator should function as an I2C slave and provide predefined button state responses.

B. Module Development

The I2C master module implements the fundamental protocol operations, including START and STOP conditions, byte level transmission, acknowledgment detection, and clock generation. The APB interface acts as a control bridge between the processor and the I2C master, allowing register configuration and transaction initiation.

The controller emulator is implemented as an I2C slave that provides predefined button state data in response to read requests. The status register or GPIO module captures the received data and makes it available for internal use or external observation.

C. Simulation and Verification

A dedicated testbench is used to generate the required control signals and initiate I2C transactions. During simulation, configuration values are written to the APB registers, read operations are triggered, FSM transitions are monitored, and the received button data is checked for correctness. Waveform analysis is used to confirm protocol compliance and timing accuracy.

D. Validation and output monitoring

The design is validated by verifying correct data acquisition from the controller emulator, accurate storage of button state information, and the absence of protocol violations during I2C and APB communication. The architecture is also suitable for FPGA implementation, where real time testing can be performed with an actual I2C based game controller.

- Writes configuration values to the APB registers
- Triggers I2C read operations
- Observes FSM transitions and bus activity
- Verifies correct reception of button state data

Waveform viewers are used to analyze signal transitions and confirm correct protocol operation.

E. Validation and Output Monitoring

After simulation, the results are analyzed to ensure the following:

- Accurate read operations from the controller emulator
- Correct mapping of button states to status registers or GPIO outputs

- No protocol violations during I2C or APB communication

The design can also be synthesized and deployed on FPGA hardware for real time testing with an actual I2C game controller.

V. FLOWCHART

The flowchart describes the transaction sequence between the SoC and the game controller over the I2C bus. The operation begins with a request for data acquisition, after which the master initiates communication by generating a START condition, transmitting the slave address with the read bit, and waiting for acknowledgment. Once acknowledgment is received, the controller data is read and stored in the internal register set before the bus is released and the system returns to the idle state.

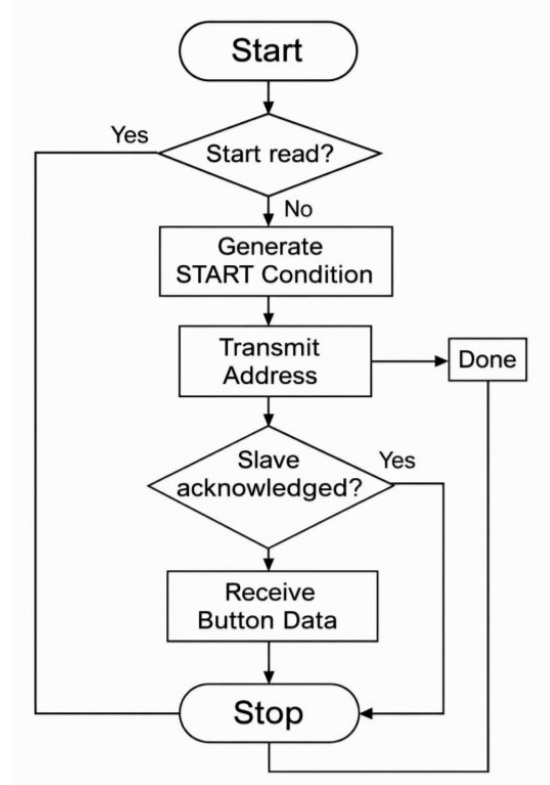


Fig. 2. Flowchart of the I2C based game controller communication process

A. Flowchart Explanation

a. Start / Initialization The process begins when the start signal is asserted. At this stage, the internal registers and counters are initialized, and the FSM transitions to the idle state.

b. Start Condition

The I2C master generates a START condition by driving SDA low while SCL remains high. This marks the beginning of a new communication transaction.

c. Send Slave Address and Read Bit

The master transmits the 7 bit slave address followed by the read bit. The address is shifted out serially on SDA in synchronization with the SCL clock.

d. Wait for ACK from Slave After the address phase, the master releases SDA and checks for acknowledgment from the slave. A low level on SDA indicates ACK, while a high level indicates NACK or absence of the slave device.

e. Read 8 bit Button Data After acknowledgment, the slave transmits 8 bits of button state information. The master samples one bit per clock cycle and stores the received value in a shift register.

f. Send ACK or NACK If more data bytes are expected, the master returns ACK after receiving the current byte. If the transfer is complete, the master generates NACK to terminate the read operation.

g. Generate Stop Condition

The master generates a STOP condition by allowing SDA to transition from low to high while SCL is high. The received data is then transferred to the button status register, and the done signal is asserted

h. Return to Idle

After transaction completion, the FSM returns to the idle state and waits for the next read request.

VI. RESULT ANALYSIS

The proposed SoC architecture was simulated to evaluate the operation of the I2C based game controller interface. The simulation confirms correct communication between the I2C master and the controller emulator modeled as an I2C slave. Button state data is successfully read and stored in the internal registers, and the protocol sequence executes without errors.

Waveform analysis shows proper synchronization between SCL and SDA throughout the transaction. Signals such as clk, rst, start, done, and data_out[7:0] exhibit the expected behavior. The results indicate that the interface supports reliable data transfer and accurate controller input capture.

A. Simulation Results

The design was verified using a dedicated testbench that initiates I2C read transactions through the APB interface. The simulation demonstrates correct generation of START and STOP conditions, successful slave address transmission, and proper reception of button state data from the controller emulator.

The observed waveforms confirm that the control and data signals behave as intended during the transaction. The I2C master correctly updates the internal registers, and no protocol violations are observed during the simulated transfer.

B. Verification Strategy

Functional verification was carried out using a simulation based testbench. The testbench generates the required control signals and monitors the responses from the controller emulator. The verification process focuses on the correctness of I2C transaction sequencing and internal data updates.

During verification, the following aspects were validated:

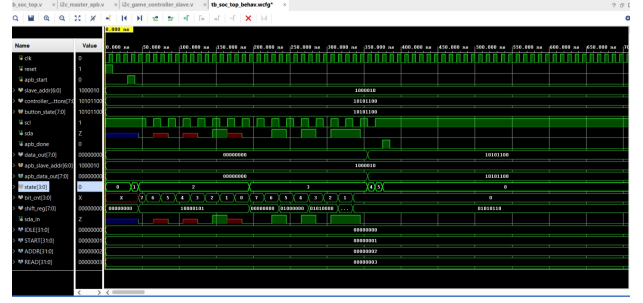


Fig. 3. Simulation waveform showing I2C communication and button data transfer

- Correct generation of START and STOP conditions during I2C communication.
- Proper transmission of the slave address and acknowledgement detection.
- Accurate reception of button state data from the controller emulator.
- Correct update of the button status register after data reception.
- Proper state transitions of the Finite State Machine (FSM).

Waveform debugging and assertion checks confirm that the design follows the I2C communication protocol without errors.

C. Latency Analysis

Latency refers to the time required for the system to capture controller input and update the internal register. In the proposed design, latency includes START generation, address transmission, acknowledgment reception, data reception, and STOP generation. Since I2C is sequential in nature, each step must complete before the next begins.

Table I shows the approximate clock cycles required for each stage of the I2C communication process.

TABLE I
LATENCY ANALYSIS OF I2C COMMUNICATION

Operation	Clock Cycles
START Condition Generation	1
Slave Address Transmission	7
Read Bit Transmission	1
ACK Reception	1
8-bit Data Reception	8
STOP Condition	1
Total Latency	19 Clock Cycles

D. Efficiency Analysis

The proposed architecture provides efficient communication by using the I2C protocol, which requires only two lines, SDA and SCL. This reduces pin count and hardware complexity compared with parallel interfaces.

In addition, the modular structure improves scalability and supports integration with other peripherals. The FSM based control logic ensures orderly transaction flow and efficient handling of communication events, making the design suitable for compact embedded systems.

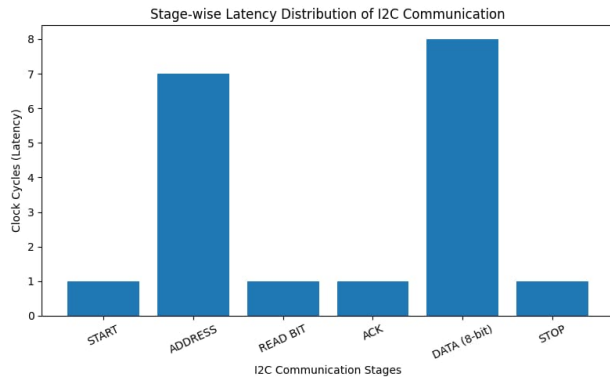


Fig. 4. Latency plot of I2C communication showing clock cycle distribution

VII. CONCLUSION

In this project, an I2C based game controller interface was designed using Verilog HDL. The main aim was to read the button data from the controller and store it inside the system. During simulation, the communication between the master and slave worked properly and the data was received correctly.

The waveform results showed that all the signals were behaving in the expected way. The latency of the system was also checked and it was found to be within a reasonable range for real time use.

Overall, the design is simple and works as intended. It can be used as a basic controller interface in embedded systems. The modular structure also makes it easier to make changes or improvements later.

VIII. FUTURE WORK

In future, this design can be improved by supporting more than one controller on the same I2C bus. Other communication protocols like SPI or UART can also be added depending on the requirement.

The design can also be tested on FPGA hardware to see how it works in real time conditions. Additional features like display output or interrupt based input handling can be included to improve the overall performance.

REFERENCES

- [1] M. Sukhanya and K. Gavaskar, "Functional verification environment for I2C master controller using SystemVerilog," *IEEE*, pp. 1–6.
- [2] S. Mehrotra and N. Charaya, "Design and implementation of I2C single master on FPGA using Verilog," *International Journal of Scientific Engineering and Research*, 2017.
- [3] M. Alassir, J. Denoulet, O. Romain, and P. Garda, "Modeling I2C communication between SoCs with SystemC-AMS," in *Proc. IEEE Int. Symp.*, 2007, pp. 1–6.
- [4] D. Kaith, J. B. Patel, and N. Gupta, "An implementation of I2C slave interface using Verilog HDL," *International Journal of Modern Engineering Research*.
- [5] A. Boddupalli, "Design and implementation of I2C bus protocol on FPGA using Verilog for EEPROM," 2014.
- [6] A. Oudjida, M. L. Berrandjia, R. Tiar, A. Liacha, and K. Tahraoui, "FPGA implementation of I2C and SPI protocols: A comparative study," in *Proc. IEEE Int. Conf.*, 2009, pp. 1–4.
- [7] M. S. L. Mukthi and A. R. Aswatha, "Design and implementation of an interfacing protocol between I2C and APB for an AMBA based SoC," *International Journal of Scientific Engineering and Science*, 2017.

- [8] P. Sai Rama Krishna and R. Bhakthavatchalu, "Optimized design of I2C master and slave controller," *IEEE*, 2025.
- [9] Z. Zhen and Z. Hui, "The hardware interface design in SoC with Verilog language," *IEEE*, pp. 1–5.
- [10] W. H. Wolf, "Hardware software co-design of embedded systems," *IEEE*, pp. 1–10.